

Services vs Infrastructure

Detailed Technical Explanation

شرح تفصيلي لدور الـ Services والـ Infrastructure في مشروع DermaVision، والفرق بينهم، وإزاي الكود بيتحرك في فلو عملي كامل من رفع الصورة لحد التشخيص وRAG والتقارير.

Practical Flow

مثال كامل يوضح مين بينادي
مين وإيه اللي بيتحفظ ويرجع للـ
.frontend

Infrastructure

التعامل مع قاعدة البيانات،
الموديل، الملفات، البريد، وLLM
.providers

Services

تنسيق business flow واتخاذ
قرارات منطق التطبيق.

1. الفكرة الأساسية: الفرق بين Services و Infrastructure

Infrastructure Layer

ال Infrastructure هو الأدوات التقنية التي تنفذ التفاصيل. يعني يجاوب سؤال: "إزاي أتعامل مع DB؟ إزاي أشغل الموديل؟ إزاي أقرأ ملف RAG؟ إزاي أبعت "؟email"

Service Layer

ال Service هو مكان ال business use case. يعني يجاوب سؤال: "إيه الخطوات المنطقية اللي لازم تحصل؟" مثال: عند رفع صورة، لازم نحفظها، نشغل الموديل، نجهز بيانات التشخيص، نحفظها في DB، ونرجع response.

تشبيه سريع: ال Service هو مدير العملية، أما Infrastructure هو العمال/الأدوات المتخصصة. المدير يقول "شخص الصورة، احفظ النتيجة"، والأدوات تنفذ: SQL insert، model inference، file save.

2. خريطة الملفات المهمة

Role	Folder / File
يدير فلو التشخيص: حفظ الصورة، تشغيل الموديل، حفظ النتيجة، history، delete.	app/domain/services/diagnosis_service.py
يتعامل مع حفظ و Validation ملفات الصور. موجود داخل services لكنه قريب من Infrastructure utility.	app/domain/services/file_service.py
ينسق RAG chat و consultation report: ownership check، intent، retrieval، generation، validation.	app/domain/services/rag_service.py
SQLAlchemy models، async session، repositories.	app/infrastructure/database
Model loading، image preprocessing، prediction engine.	app/infrastructure/ml
RAG tools: document loader، splitter، vector store، retriever، generator، providers، safety profiles.	app/infrastructure/rag
SMTP email sender for password reset.	app/infrastructure/email_service.py
يركب dependencies: current user، DB session، repositories، services.	app/api/dependencies.py

3. كيف FastAPI يتركب ال Services؟

في `app/api/dependencies.py` يوجد endpoint `Dependency Injection`. لا ينشئ كل شيء بنفسه؛ FastAPI يحقن له الخدمة جاهزة.

```
get_diagnosis_service(  
    predictor = SkinDiseasePredictor(),  
    file_service = FileService(),  
    diagnosis_repo = DiagnosisRepository(db)  
) -> DiagnosisService
```

هذا معناه أن `DiagnosisService` لا يعتمد على endpoint مباشرة، بل يعتمد على مكونات واضحة: `file`، `predictor`، `repository`، `service`. وهذا يجعل الكود أسهل في الفهم والاختبار.

الميزة: لو غيرت طريقة التخزين أو غيرت موديل ML، غالبًا هتغير Infrastructure فقط، بينما Service flow يظل ثابت.

4. DiagnosisService: دوره وإزاي الكود بيشتغل

DiagnosisService هو أهم Service في فلو التشخيص. هو لا يعرف تفاصيل SQL ولا تفاصيل EfficientNet الداخلية؛ هو فقط ينظم الخطوات.

Dependencies التي يستخدمها

Dependency	Type	Purpose
SkinDiseasePredictor	Infrastructure ML	تشغيل inference على الصورة وإرجاع prediction.
FileService	File utility service	حفظ الصورة والتحقق منها وحذفها عند الفشل أو الحذف.
DiagnosisRepository	Infrastructure DB	حفظ وقراءة وحذف diagnosis من قاعدة البيانات.

create_diagnosis ماذا تفعل؟



Other methods

- `get_diagnosis(diagnosis_id, user_id)` : يقرأ التشخيص ويتأكد أنه يخص نفس المستخدم.
- `get_user_history(user_id)` : يرجع history الخاصة بالمستخدم.
- `delete_diagnosis(diagnosis_id, user_id)` : يتحقق من الملكية، يحذف الصورة من storage، ثم يحذف record من DB.

5. FileService: دوره وإزاي الكود بيشتغل

FileService مسؤول عن التعامل الآمن مع ملفات الصور. هو موجود في domain/services، لكن وظيفته أقرب لـ technical file infrastructure.

Method	What it does
save_upload	يتحقق من الملف، ينشئ اسم آمن، يحفظ الصورة، ثم يتأكد أنها صورة حقيقية.
_validate_file	يتأكد أن الملف غير فارغ، حجمه أقل من 10MB، وامتداده JPG/JPEG/PNG.
_validate_image_content	يستخدم Pillow للتحقق أن الملف صورة حقيقية وليس ملف مزيف بامتداد صورة.
delete_file	يحذف صورة محفوظة عند حذف التشخيص أو عند فشل العملية.
cleanup_old_files	ينظف ملفات قديمة بناءً على عدد أيام.

نقطة Security مهمة: الكود لا يثق في اسم الملف القادم من المستخدم. يستخدم session_id أو UUID ويعمل resolve للمسار لمنع path traversal.

6. RagService: دوره وإزاي الكود بيشتغل

RagService هو Service معقد لأنه يجمع بين قاعدة البيانات، تشخيص المستخدم، LLM، retrieval، intent detection، provider، وmedical safety.

chat method flow



final_report method

- يأخذ analysis_id وmessages من الشات.
- يتحقق من ملكية التشخيص.
- يستخدم disease profile لتجهيز تقرير آمن.
- يلخص أسئلة المستخدم: diagnosis، danger، contagion، treatment، children/adults، dosage.
- يرجع summary عامة، guidance، treatment options، red flags، disclaimer.

7. Infrastructure: Database Layer

Database infrastructure موجود في `app/infrastructure/database`. دوره ليس اتخاذ business decision، بل التعامل مع SQLAlchemy والجداول وال sessions.

File	Role
<code>base.py</code>	يعرف Base المشترك لكل SQLAlchemy models.
<code>session.py</code>	ينشئ async engine و session dependency للتعامل مع DB.
<code>models/user.py</code>	جدول users: email, username, hashed_password, timestamps
<code>models/diagnosis.py</code>	جدول diagnoses: user_id, image_path, disease_type, probability, quality, predictions
<code>models/family_member.py</code>	جدول family_members وريبطه بالمستخدم والتشخيصات.
<code>models/password_reset_token.py</code>	جدول reset tokens: token_hash, expires_at, used_at
<code>repositories/diagnosis_repository.py</code>	CRUD لل diagnosis بدون كشف SQL details لل services.
<code>repositories/user_repository.py</code>	قراءة/إنشاء users وتحديث password/deactivate

مثال Repository Pattern

بدل ما `DiagnosisService` يكتب SQLAlchemy queries بنفسه، هو ينادي:

```
diagnosis = await diagnosis_repo.create(diagnosis_data)
history = await diagnosis_repo.get_by_user(user_id=user_id)
diagnosis = await diagnosis_repo.get_by_id(analysis_id)
```

بهذا الشكل ال Service يظل مركز على business logic، وال Repository مسؤول عن SQL.

Infrastructure: ML Layer .8

ML infrastructure موجود في `app/infrastructure/ml` . دوره تشغيل الموديل وليس اتخاذ قرار `business flow`.

File / Class	Role
ModelLoader	Singleton مسؤول عن تحميل EfficientNet-B4 weights مرة واحدة، اختيار CPU/CUDA، warmup، health status
SkinDiseasePredictor	Prediction engine: يفتح الصورة، يحسب الجودة، يعمل preprocessing، يشغل inference، ويرجع top_k
ImagePreprocessor	Normalize، ToTensor، Resize، وتقييم مبسط لجودة الصورة حسب resolution.
constants.py	أسماء الأمراض، mean/std، image size
weights/skin_efficientnet_b4_9.pth	ملف أوزان الموديل المدرب.

لماذا ModelLoader Singleton؟ لأن تحميل الموديل ثقيل. لا نريد تحميل weights مع كل request، بل مرة واحدة ثم نعيد استخدام نفس instance.

Infrastructure: RAG Layer .9

knowledge RAG infrastructure هي الأدوات الصغيرة التي يستخدمها RagService لبناء إجابة آمنة ومبنية على .base

Role	Module
تحميل ملف المعرفة skin_data1.txt وإرجاع + text .metadata	document_loader.py
تقسيم الملف إلى chunks مع disease tags.	text_splitter.py
فهرس محلي TF-IDF-like يخزن chunks ويبحث فيها.	vector_store.py
يعمل retrieval وreranking حسب المرض المتوقع والintent.	retriever.py
كشف local intents: diagnosis, symptoms, treatment, contagion, urgent, dosage, age groups.	intent.py
تصنيف intent اختياريًا باستخدام Groq/xAI/OpenAI إذا provider متاح.	semantic_intent_classifier.py
مولد deterministic fallback آمن بدون LLM key.	generator.py
LLM providers باستخدام OpenAI-compatible clients.	xai_generator.py , groq_generator.py , openai_generator.py
canonical disease names و safety facts مثل Urticaria non-contagious و Tinea contagious.	disease_profiles.py

10. Infrastructure: Email Service

email_service.py مسؤول عن SMTP password reset email. هو Infrastructure لأنه يتعامل مع خدمة خارجية SMTP.

- `is_configured()` : يتأكد من وجود SMTP_HOST و SMTP_FROM_EMAIL وأن username/password متطابقين في الوجود.
- `send_password_reset_email()` : ييني email message فيه reset link، مدة الصلاحية، وتنبيه تجاهل الرسالة إذا لم يطلبها المستخدم.
- لو SMTP غير مهيا، في development يمكن استخدام reset link من backend terminal، لكن في production لا يتم كشفه للمستخدم.

EmailService لا يقرر من له حق Auth endpoint. reset password هو الذي يقرر ذلك. EmailService فقط "بيعت email".

11. مثال عملي كامل: Upload Image Diagnosis Flow

هذا مثال فلو كامل يوضح مين بينادي مين عندما المستخدم يرفع صورة من ال frontend.

1

Frontend Upload

المستخدم يختار صورة
ويضغط Analyze.

2

API Request

Frontend يرسل
POST
/api/v1/diagnosis
مع JWT.

3

Auth Dependency

get_current_user_id
يفك JWT ويرجع user_id.

4

Endpoint Validation

يتحقق من MIME type
وحجم الملف.

5

Service Injection

FastAPI يجهز
DiagnosisService.

6

create_diagnosis

Service يبدأ business
flow.

7

FileService

يحفظ الصورة باسم آمن
ويتحقق بمكتبة Pillow.

8

Predictor

يفتح الصورة ويحولها RGB
ويجهز tensor.

9

ModelLoader

يعطي الموديل المحمل
EfficientNet-B4.

10

Inference

PyTorch يعمل softmax
ويرجع top_k.

11

Prepare DB data

Service ييني
diagnosis_data.

12

Repository

يحفظ record في جدول
diagnoses.

13

Mapper

API mapper يحول
النتيجة لشكل مناسب لل-
frontend.

14

Result Page

Frontend يعرض
diagnosis, confidence,
image quality, report
actions.

15

History

التحليل أصبح متاحًا في
Dashboard/History.

16

Error cleanup

لو حصل خطأ بعد حفظ
الصورة، Service يحذف
الملف.

12. مثال عملي كامل: RAG Chat Flow

بعد ظهور النتيجة، المستخدم يمكنه فتح `/chat?analysisId=123` وسؤال المساعد.



13. مثال عملي: Consultation Report Flow

- المستخدم يتحدث مع المساعد عن المرض والخطورة والعدوى والعلاج العام.
- Frontend يجمع chat messages من state/sessionStorage.
- يرسل POST /api/v1/rag/final-report مع analysis_id و messages و language.
- RagService.final_report يحمل diagnosis ويتأكد من ownership.
- يستخدم disease_profiles.py لبناء حقائق آمنة حسب المرض.
- يلخص أسئلة المستخدم في bullets بدل نسخ المحادثة كاملة.
- يرجع report sections: diagnosis, summary, patient concerns, care guidance, treatment options عامة, red flags, disclaimer.
- Frontend يعرض report modal مع Print/Copy.

التقرير ليس وصفة علاجية. هو Consultation Report تعليمي يساعد المستخدم يعرف ماذا يناقش مع الطبيب.

14. لماذا هذا الفصل بين Services و Infrastructure مهم؟

Testability

يمكن اختبار Service بمنطق وهمي/mock predictor أو mock repository.

Maintainability

لو غيرنا SQLite إلى PostgreSQL، غالبًا التغيير في infrastructure/database، وليس في DiagnosisService flow.

Flexibility

يمكن تبديل Groq/xAI/OpenAI/local fallback بدون تغيير endpoint contract.

Security

ownership checks والقرارات الحساسة تبقى واضحة في service/API layer، بدل ما تتوزع في كل مكان.

Presentation Value

في المناقشة تقدر تقول إن المشروع ليس كود عشوائي، بل مبني على clean architecture وpatterns واضحة.

Clear Responsibilities

كل ملف يعرف دوره: service ينظم، repository يحفظ، predictor يشخص، retriever يبحث، generator يرد.

15. جملة مختصرة للمناقشة

"في DermaVision فصلت الـ business logic عن التفاصيل التقنية. الـ Services مثل DiagnosisService و RagService تنسق الفلو وتطبق قواعد النظام، بينما Infrastructure مثل database repositories و ML predictor و RAG retriever و email service تنفذ التفاصيل التقنية. هذا يجعل المشروع قابل للصيانة، الاختبار، والتطوير بدون كسر الـ API."

مثال سريع تقوله

"عند رفع صورة، endpoint يستقبل الطلب ويتحقق من JWT، ثم ينادي DiagnosisService. الـ Service يحفظ الصورة عن طريق FileService، يشغل الموديل عن طريق SkinDiseasePredictor، ثم يحفظ النتيجة عن طريق DiagnosisRepository. هنا Service هو من يدير الفلو، و Infrastructure هو من ينفذ الأدوات."

Final Summary .16

- **.Services** = orchestration + business rules + ownership/safety decisions
- **.Infrastructure** = technical implementation: DB, ML model, files, RAG index, email, LLM providers
- **DiagnosisService** يدير فلو التشخيص من الصورة إلى DB response.
- **RagService** يدير الشات والتقرير مع retrieval و guardrails.
- **Infrastructure modules** تجعل الكود modular وقابل للتغيير بدون تغيير الفلو الرئيسي.